

# Standard SQL Functions (PostgreSQL, Oracle, MySQL 8+, Snowflake)

Cross-RDBMS

| PATTERN / SYNTAX                                                                 | DESCRIPTION & EXAMPLE                                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>REGEXP_LIKE(str, pattern [, flags])</code>                                 | Returns TRUE/1 if regular expression matches anywhere in str; supported in Oracle, MySQL 8+, Snowflake, Databricks<br><br>EXAMPLE: <code>SELECT * FROM users WHERE REGEXP_LIKE(email, '^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}\$', 'i');</code> |
| <code>REGEXP_SUBSTR(str, pattern [, pos [, occ [, flags [, group]]])</code>      | Extracts the matched substring (or specific capture group) from str; pos defaults to 1, occ defaults to 1<br><br>EXAMPLE: <code>SELECT REGEXP_SUBSTR('invoice_2026_08.pdf', '\d{4}') → '2026'</code>                                           |
| <code>REGEXP_REPLACE(str, pattern, replacement [, pos [, occ [, flags]]])</code> | Replaces matched substrings in str with replacement text (supports backreferences \1, \2 or \$1, \$2)<br><br>EXAMPLE: <code>SELECT REGEXP_REPLACE('phone: 555-1234', '\d', 'X') → 'phone: XXX-XXXX'</code>                                     |
| <code>REGEXP_INSTR(str, pattern [, pos [, occ [, return_opt [, flags]]])</code>  | Returns 1-based start position of the N-th match (or position after match if return_opt = 1); 0 if no match<br><br>EXAMPLE: <code>SELECT REGEXP_INSTR('Order #4092 created', '\d+') → 8</code>                                                 |
| <code>REGEXP_COUNT(str, pattern [, pos [, flags]])</code>                        | Returns total count of non-overlapping matches in str (PostgreSQL 15+, Oracle, Snowflake, MySQL 8+)<br><br>EXAMPLE: <code>SELECT REGEXP_COUNT('cat, dog, bird, fish', '\w+') → 4</code>                                                        |

# PostgreSQL Regex Operators & Native Functions

| PATTERN / SYNTAX                                           | DESCRIPTION & EXAMPLE                                                                                                                                                                                           |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>str ~ pattern</code>                                 | <p>POSIX regular expression match operator (case-sensitive) — returns boolean true/false</p> <p>EXAMPLE: <code>SELECT '2026-08' ~ '^d{4}-d{2}\$' → true</code></p>                                              |
| <code>str ~* pattern</code>                                | <p>POSIX regular expression match operator (case-insensitive) — returns boolean true/false</p> <p>EXAMPLE: <code>SELECT 'Alice' ~* '^a.*e\$' → true</code></p>                                                  |
| <code>str !~ pattern / str !~* pattern</code>              | <p>POSIX regular expression non-match operators (case-sensitive !~ and case-insensitive !~*)</p> <p>EXAMPLE: <code>SELECT 'hello' !~* 'd' → true</code></p>                                                     |
| <code>REGEXP_MATCH(str, pattern [, flags])</code>          | <p>Extracts first match as a text[] array of captured groups (or entire match if no groups); returns NULL if no match</p> <p>EXAMPLE: <code>SELECT (REGEXP_MATCH('id: 420', 'id: (\d+)'))[1] → '420'</code></p> |
| <code>REGEXP_MATCHES(str, pattern [, flags])</code>        | <p>Returns set of text[] arrays for all matches (use 'g' flag for global multi-row set returning function)</p> <p>EXAMPLE: <code>SELECT REGEXP_MATCHES('a1 b2 c3', '([a-z])(\d)', 'g');</code></p>              |
| <code>REGEXP_SPLIT_TO_TABLE(str, pattern [, flags])</code> | <p>Splits string by regex matches and returns each item as a separate table row (set of text)</p> <p>EXAMPLE: <code>SELECT REGEXP_SPLIT_TO_TABLE('apple,banana;cherry', '[,;]\s*');</code></p>                  |
| <code>REGEXP_SPLIT_TO_ARRAY(str, pattern [, flags])</code> | <p>Splits string by regex matches into a single PostgreSQL array (text[])</p> <p>EXAMPLE: <code>SELECT REGEXP_SPLIT_TO_ARRAY('10.20.30', '\. ') → '{10,20,30}'</code></p>                                       |
| <code>str SIMILAR TO pattern</code>                        | <p>SQL standard pattern matching using SQL99 rules (combines LIKE wildcards % _ with regex   * + ( ))</p> <p>EXAMPLE: <code>SELECT 'abc' SIMILAR TO '%(b d)%' → true</code></p>                                 |

# MySQL, MariaDB, & SQLite

| PATTERN / SYNTAX                                                            | DESCRIPTION & EXAMPLE                                                                                                                                                                      |
|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>str REGEXP pattern / str RLIKE pattern</code>                         | Matches regular expression anywhere within str; RLIKE is an exact synonym of REGEXP in MySQL and MariaDB<br><br>EXAMPLE: <code>SELECT 'Product 99' REGEXP '[0-9]+' → 1</code>              |
| <code>str NOT REGEXP pattern / str NOT RLIKE pattern</code>                 | Returns 1 if pattern does not match str, 0 if it matches, and NULL if either argument is NULL<br><br>EXAMPLE: <code>SELECT 'abc' NOT REGEXP '\d' → 1</code>                                |
| <code>REGEXP_LIKE(expr, pat [, match_type])</code>                          | MySQL 8.0+ functional syntax with match_type flags ('i', 'c', 'm', 'n', 'u')<br><br>EXAMPLE: <code>SELECT REGEXP_LIKE('Admin_User', '^admin', 'i') → 1</code>                              |
| <code>REGEXP_SUBSTR(expr, pat [, pos [, occurrence [, match_type]]])</code> | MySQL 8.0+ function returning matched substring, or NULL if no match found<br><br>EXAMPLE: <code>SELECT REGEXP_SUBSTR('item #429 in stock', '\d+') → '429'</code>                          |
| <code>str REGEXP pattern (SQLite)</code>                                    | SQLite regex operator — requires the regexp() extension function to be loaded/registered by the host driver<br><br>EXAMPLE: <code>SELECT * FROM items WHERE name REGEXP '^A[0-9]+';</code> |

# Cloud Data Warehouses (Snowflake, BigQuery, Databricks, Trino)

| PATTERN / SYNTAX                                                                 | DESCRIPTION & EXAMPLE                                                                                                                                                                                              |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>REGEXP_CONTAINS(value, regex)</code><br>(BigQuery)                         | Returns TRUE if regex finds a match anywhere in string value in Google BigQuery<br><br>EXAMPLE: <code>SELECT REGEXP_CONTAINS(url, r'https?:/[a-z0-9.-]+')</code> FROM <code>web_traffic</code> ;                   |
| <code>REGEXP_EXTRACT(value, regex [, position [, occurrence]])</code> (BigQuery) | Extracts the first matching substring or first capturing group from value in BigQuery<br><br>EXAMPLE: <code>SELECT REGEXP_EXTRACT('order_id: 9941', r'\d+') → '9941'</code>                                        |
| <code>REGEXP_EXTRACT_ALL(value, regex)</code><br>(BigQuery / Trino)              | Returns an ARRAY of all non-overlapping matches or capture groups in BigQuery and Trino/Presto<br><br>EXAMPLE: <code>SELECT REGEXP_EXTRACT_ALL('v1.0, v2.1, v3.5', r'v\d+\.\d+') → ['v1.0', 'v2.1', 'v3.5']</code> |
| <code>REGEXP_LIKE(str, pattern [, parameters])</code> (Snowflake)                | Evaluates regex match against string column with parameter flags in Snowflake / Databricks<br><br>EXAMPLE: <code>SELECT * FROM sales WHERE REGEXP_LIKE(sku, '^[A-Z]{3}-\d{4}\$');</code>                           |
| <code>str RLIKE pattern</code> (Spark / Databricks / Hive / Trino)               | Tests if str matches pattern regular expression using Java regex syntax under the hood<br><br>EXAMPLE: <code>SELECT * FROM events WHERE event_name RLIKE 'user_(login logout)\$';</code>                           |

# SQL Match Parameters & Flag Modifiers

| PATTERN / SYNTAX | DESCRIPTION & EXAMPLE                                                                                                                                                                         |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'i'              | Case-insensitive matching parameter (e.g. REGEXP_LIKE(name, '^john', 'i'))<br><br>EXAMPLE: REGEXP_LIKE('JOHN', 'john', 'i') → TRUE                                                            |
| 'c'              | Case-sensitive matching parameter (default in most SQL engines except case-insensitive collations)<br><br>EXAMPLE: REGEXP_LIKE('JOHN', 'john', 'c') → FALSE                                   |
| 'm'              | Multiline mode — anchors ^ and \$ match beginning and end of each physical line within string<br><br>EXAMPLE: REGEXP_REPLACE(multiline_text, '^#s*', '', 1, 0, 'm')                           |
| 'n' / 's'        | Singleline / dot-all mode — permits dot (.) to match newline characters (flags vary: 'n' in Oracle/Postgres, 's' in PCRE)<br><br>EXAMPLE: REGEXP_SUBSTR(html_doc, '<div>.*</div>', 1, 1, 'n') |
| 'x'              | Extended/verbose mode — ignores whitespace and allows # comments inside regex pattern string<br><br>EXAMPLE: REGEXP_LIKE(val, '^\\d{4} # Year \\n-\\d{2} # Month', 'x')                       |
| 'g' (PostgreSQL) | Global replacement / match parameter in PostgreSQL (replaces or matches all occurrences)<br><br>EXAMPLE: REGEXP_REPLACE('a1 b2 c3', '\\d', 'X', 'g') → 'aX bX cX'                             |

# SQL Server (T-SQL) LIKE Character Ranges & Wildcards

| PATTERN / SYNTAX                      | DESCRIPTION & EXAMPLE                                                                                                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>LIKE '[0-9]'</code>             | Matches any string containing at least one digit character (0 through 9) in T-SQL<br><br>EXAMPLE: <code>SELECT * FROM orders WHERE order_num LIKE 'ORD-[0-9][0-9][0-9][0-9]';</code> |
| <code>LIKE '[^0-9]'</code>            | Negated set — matches any string containing at least one character that is NOT a digit<br><br>EXAMPLE: <code>SELECT * FROM users WHERE zip_code LIKE '[^0-9]';</code>                |
| <code>LIKE '[A-Za-z]'</code>          | Matches strings starting with any alphabetic letter (A-Z or a-z)<br><br>EXAMPLE: <code>SELECT * FROM customers WHERE name LIKE '[A-M]';</code>                                       |
| <code>PATINDEX('[0-9]%', str)</code>  | Returns 1-based start position of first occurrence of pattern in str; returns 0 if not found<br><br>EXAMPLE: <code>SELECT PATINDEX('[0-9]%', 'Item #42') → 7</code>                  |
| <code>LIKE '[%_]' (ESCAPE '\')</code> | Escapes literal % or _ wildcard characters using custom ESCAPE clause<br><br>EXAMPLE: <code>SELECT * FROM logs WHERE message LIKE '%100\%' ESCAPE '\';</code>                        |